

Contents

Overview	2
Shell Card Transaction Data API.....	2
The Shell Developer Hub.....	2
API Endpoints.....	3
Environments	3
Authentication	3
Pagination	3
Rate Limits.....	4
Error Codes	4
Request Ids.....	5
Best Practices	5
FAQs	6



Quick Start Guide – Shell Card Transaction Data API

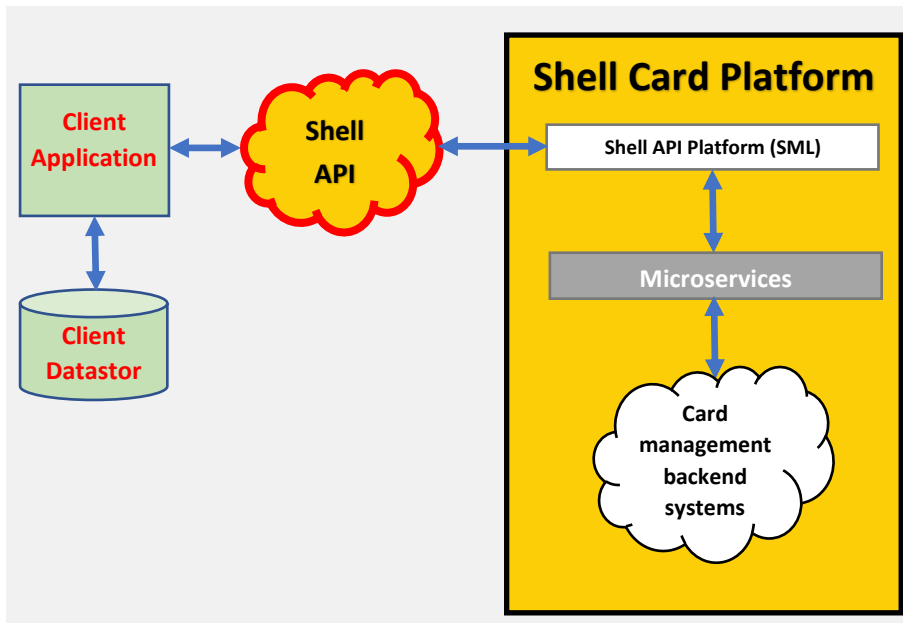
Overview

The Shell Card Transaction Data API allows access to transactions data performed by Shell fuel cards. The purpose of this guide is to provide an overview of the available API endpoints and an introduction to some of the fundamental areas of working with the APIs.

Shell Card Transaction Data API

The Shell Card Transaction Data API is REST-based, which allows clients to initiate requests against the Shell Card Platform. The Shell API endpoints accept JSON-encoded request bodies, return JSON-encoded responses, and use standard HTTP response codes and authentication.

The typical components of a basic integration using the Shell REST API are shown below.



Some of the key components include:

- **Client Application and Database:** The Client Application is the integration system, which uses the Shell API to integrate data. The development, maintenance and support of the client application is the responsibility of the customer.
- **Shell Card Platform:** The Shell Card Platform is the overall platform that encompasses all the internal Shell systems used to manage fuel cards. The internal workings of the platform are not important when interacting with the API. However, it is worth noting a microservice architecture is used to communicate with the backend systems.
- **Shell API Platform (SML):** This is the Service Mediation Layer (SML) within the Shell Card Platform that manages API authentication and makes Shell fuel card data available through public facing API endpoints.
- **Backend systems:** There are various backend systems used to manage and store Shell fuel card data. Calls to the backend can either be synchronous or asynchronous with the microservice layer queuing data to the backend services.

The Shell Developer Hub

The developer hub is located online at <https://apiportal.shell.com>. The hub provides all the current API product offerings and detailed API specifications for each product.

We recommend referencing the full API specifications for the Shell Card Transaction Data API alongside the quick start guides.



Quick Start Guide – Shell Card Transaction Data API

API Endpoints

The API endpoints are functionally classified into the areas below. Each endpoint belongs to a commercial package that determines whether it is available for an API client and the number of API calls that can be executed per minute. Further details on handling call limits can be found in the section 'Rate Limits'.

Functional Area	API Endpoint	Description	Commercial Package
Search transactions	/transaction/pricedtransactions	Get priced transactions for a single payer	Data & Reporting Starter
	/transaction/multipayerspricedtransactions	Get priced transactions for multiple payers	Data & Reporting Starter
	/transaction/pricedtransactionssummary	Get priced transaction summary	Data & Reporting Advanced
Search fees	/transaction/fees	Get fee/charges levied from customers account for bill period	Data & Reporting Starter
	/transaction/feesummary	Get fee items summary by fee group	Data & Reporting Starter

Environments

The Shell Card Transaction Data API is available on the following environments. All examples in the quick start guides reference the test environment.

Environment	Base URL	Usage
Test	https://api-test.shell.com/test/	Development User Acceptance Testing (UAT)
Production	https://api.shell.com/	Smoke testing

	Live integration
--	------------------

Authentication

The Shell APIs are authorised with an API key and HTTP Basic Authentication. A customer is identified as an API client and is provided a unique API key and secret for each of the supported environments.

All Shell API endpoints require the default authorisation header including the target *host*, *apikey* and a base64 encoded *Authorization* header. The *Authorization* header contains the word *Basic* followed by a base64 encoded value of *apiKey:secret*. For example:

```
Host: api-test.shell.com/test
Content-Type: application/json
Authorization: Basic RXZyQWhTeFpYcDhRQVBSSUp2bmthOnNlY3JldA==
apikey: EvrAhSxZXp8QAPRIJvnka
```

Pagination

Pagination is available for endpoints that return large amounts of data. The number of resources returned by an API call can be set by including the optional *PageSize* field in the request. The default value is 50 if the field is not provided. Setting the value to -1 will return all resources but this is generally not recommended as there may be performance implications for requesting large amounts of data in a single call.

The presence of the following fields in an API response indicates the support for pagination:

Field Name	Description
CurrentPage	In the response, indicates which page is being returned. In the request, indicates which page to request.



Quick Start Guide – Shell Card Transaction Data API

RowCount	In the response, indicates the number of resources returned per page.
TotalPages	In the response, indicates the total number of pages available,
PageSize	In the request, indicates the number of resources to return per page.

Rate Limits

The rate limits for API calls are on a per minute basis. The limits are determined by the commercial package assigned to the API client. The default minimum limit starts at 10 API calls per minute.

Regardless of the package limits, once the limit has been exceeded, the API response will return an error code *'policies.ratelimit.QuotaViolation'*.

```
{
  "fault": {
    "faultstring": "Rate limit quota violation. Quota limit exceeded. Identifier: EvrAhSxZXp8QAPRIJvnka",
    "detail": {
      "errorcode": "policies.ratelimit.QuotaViolation"
    }
  }
}
```

It is recommended integrations handle this error code gracefully and reattempt API calls after a duration of a minute.

Error Codes

The API calls return error codes at two levels. Errors capture at the SML layer will typically be non 200 http status codes. If the SML accepts an API call for the requesting API user, it then passes the request to the Shell Card Platform. The platform will process the request and return validation or other processing errors.

The following table illustrates the typical error responses at each level. The full list of standard error codes are documented in the API specification in the developer hub.

Error level	HTTP status code	Response body
Shell API Platform (SML)	400, 401, 404, 500 and other standard HTTP status codes indicating failure	<pre>{ "fault": { "faultstring": "Invalid ApiKey for given resource", "detail": { "errorcode": "oauth.v2.InvalidApiKeyForGivenResource" } } }</pre>
Shell Card Platform	200	The http status code 200 and the <i>Error.Code</i> "0000" in the response body indicates the API call succeeded. <pre>"Error": { "Code": "0000", "Description": "Success" }</pre> The http status code 200 with Error Code other than "0000" in the response body indicates the API call failed. <pre>"Error": { "Code": "0007", "Description": "Invalid parameter value - [Status]" }</pre>
	400	The Shell Card Management Platform understood the request but refuses to authorize it. <pre>{</pre>



Quick Start Guide – Shell Card Transaction Data API

```
"Error": {  
  "Code": "0002",  
  "Description": "User does not have a  
ccess to one or more payers and/or accounts  
/ Logged in user does not have access to the  
requested operation."  
}
```

Request Ids

A unique request identifier in the form of a GUID can be assigned to API requests. This can be provided by an API client, otherwise the Shell Card Platform will automatically assign a GUID where a value is not provided.

The API response will return the request Id value for all successful http status 200 calls regardless of whether the Shell Card Platform accepted or rejected the request (e.g. validation error). The request Id can be provided to the Shell Technical Support team, when requesting additional support for troubleshooting. This will help the team investigate in a timely manner.

Best Practices

The following best practice guidelines are recommended for API client applications and integrations.

API	Recommendations
Transactions /PricedTransactions	This API is intended for the clients to obtain the list of transactions for the given search criteria. It is recommended to use paging for presenting an interactive transactions list

API	Recommendations
	- unless the user is attempting to download the whole list of transactions into a file. - Recommended Page Size: <50
Transactions /PricedTransactionsSummary	- This API is intended to serve the summary of transactions for the given search criteria of the “PricedTransactions” API. This will help the client to present the summary of the search operation being made - The API output should be cached and reused while the user is navigating across the pages. This API should not be called for every page of the PricedTransactions API call. This should be called only if there is a change to the search parameters
General	- Where PayerID, AccountID or Card ID are allowed on the input parameters of the API, Clients must consider passing these ID’s if known to them as this will have better performance. These Id’s are also returned on most of API’s response to the clients so the clients where possible can make use them on the subsequent API calls. If the Id’s are not known, Clients must pass a valid Payer Number, Account Number and PAN (Natural keys) - Avoid logging/storing full PAN numbers or other sensitive information to the disk. Consider masking or storing related Id’s if needed.



Quick Start Guide – Shell Card Transaction Data API

FAQs

Do I need an account to view the API specification on the Shell Developer Hub?

No, the specifications for the Shell Card Management are publicly available at <https://apiportal.shell.com/api/shell-card-transaction-data-api>

Do I need to provide a RequestId value when making an API call?

All API endpoints accept an optional RequestId input parameter in the request body. This is a GUID passed in from an API client or automatically populated by the Shell Card Platform where a value is not provided.

If you need to contact the Shell Technical Support team, then providing the request identifier value will make it easier to investigate in a timely manner.

Do Shell API calls timeout?

Yes, if the Shell API Platform is unable to process an API call within 55 seconds then it will timeout. A http status code 504 will be returned.

